

UNIVERSITATEA TEHNICĂ A MOLDOVEI

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

LUCRARE DE LABORATOR NR. 6

Inside the xv6 File System

Disciplina: Sisteme de Operare

A efectuat:
student gr. FAF-243
Daniel Nicorici

Au verificat:
asist. univ. Patricia Reițman
asist. univ. Marius Dumitrașcu

Chișinău — 2026

Cuprins

Purpose of the work **2**

1 Part 1. Read what xv6 shows you **2**

2 Part 2. Write mystat **3**

3 Part 3. Measure the limits **6**

4 Part 4. Write find **7**

Conclusions **9**

Purpose of the work

The xv6 file system uses the same ideas as ext4 (inodes, directories stored as tables of entries, hard links and a journal), but in a few hundred readable lines. The purpose of this lab is to read what xv6 shows about its files, to write a `mystat` program that prints an inode's fields and use it to watch hard links at work, to measure the limits defined in `kernel/fs.h`, and to write `find`, which walks the directory tree recursively.

The work was done in my `~/xv6-riscv` tree (`mit-pdos/xv6-riscv`, commit `06aad25`), cross-compiled with `riscv64-linux-gnu-gcc` 16.2.1 and run on QEMU 11.1.1 on my Arch Linux laptop. Every experiment was run on a fresh disk image. Each new program is a file `user/NAME.c` plus a line `$U/_NAME\` in `UPROGS`.

1 Part 1. Read what xv6 shows you

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35904
echo           2 4 34800
forktest       2 5 17704
grep           2 6 43328
init           2 7 35144
kill           2 8 34696
ln             2 9 34496
ls             2 10 41984
mkdir          2 11 34752
rm             2 12 34728
sh             2 13 57104
stressfs       2 14 35576
usertests      2 15 201552
grind          2 16 50376
wc             2 17 36872
zombie         2 18 34048
logstress      2 19 36720
forphan        2 20 35512
dorphan        2 21 34968
sync           2 22 34144
sleep          2 23 34472
args           2 24 34464
mystat         2 25 35424
find           2 26 41336
console        3 27 0
```

Figure 1.1 – ls on a fresh xv6 disk

Each line is: name, type, inode number and size in bytes. My disk also contains the programs from earlier labs (`sleep`, `args`) and this lab (`mystat`, `find`), so `console` has inode 27 instead of the 25 shown in the assignment.

```
$ mkdir a
$ ls a
.              1 28 32
```

```
..              1 1 1024
```

Figure 1.2 – An empty directory

```
#define T_DIR    1 // Directory
#define T_FILE   2 // File
#define T_DEVICE 3 // Device

struct stat {
  int dev;     // File system's disk device
  uint ino;    // Inode number
  short type;  // Type of file
  short nlink; // Number of links to file
  uint64 size; // Size of file in bytes
};
```

Figure 1.3 – File types and struct stat in kernel/stat.h

```
#define DIRSIZ 14

// The name field may have DIRSIZ characters and not end in a NUL
// character.
struct dirent {
  ushort inum;
  char name[DIRSIZ] __attribute__((nonstring));
};
```

Figure 1.4 – A directory entry in kernel/fs.h

Observations:

1. **Types 1, 2 and 3.** From `kernel/stat.h`: 1 is `T_DIR`, a directory; 2 is `T_FILE`, a regular file; 3 is `T_DEVICE`, a device. `console` is the only device, the terminal, exactly like `/dev/tty` on Linux.
2. **The root directory.** The root has inode number 1. Both `.` and `..` show inode 1 because the root has no parent: when `mkfs` creates the disk, it makes the root's `..` point back to the root itself, so going "up" from `/` stays in `/`.
3. **Size of a directory entry.** A directory is just a file filled with `struct dirent` records: a 2-byte inode number (`ushort inum`) and a 14-byte name (`DIRSIZ`), 16 bytes in total. The empty directory `a` has size 32, which is two entries: `.` pointing to `a` itself (inode 28) and `..` pointing to its parent, the root (inode 1).

2 Part 2. Write mystat

`stat()` asks the kernel to fill a `struct stat` for a path. The program converts the type number into a word with a `switch`, prints an error to file descriptor 2 for a name that does not exist and continues with the next one, and prints the four fields for every name that does.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
```

```
typename(int t)
{
  switch(t){
  case T_DIR:
    return "directory";
  case T_FILE:
    return "file";
  case T_DEVICE:
    return "device";
  }
  return "?";
}

int
main(int argc, char *argv[])
{
  struct stat st;
  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
```

Figure 2.1 – user/mystat.c

```
$ grep -n "_mystat\|_find" Makefile
155:    $U/_mystat\
156:    $U/_find\
```

Figure 2.2 – The UPROGS lines for mystat and find

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 27, links 1, size 0 bytes
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
48 336 2441
$ mystat README nosuchfile
mystat: cannot stat README
mystat: cannot stat nosuchfile
```

Figure 2.3 – mystat watching a hard link

```
  memset(&de, 0, sizeof(de));
```

```
236   if (writei(dp, 0, (uint64)&de, off, sizeof(de)) != sizeof(de))
237     panic("unlink: writei");
238   if (ip->type == T_DIR) {
239     dp->nlink--;
240     iupdate(dp);
241   }
242   iunlockput(dp);
243
244   ip->nlink--;
245   iupdate(ip);
246   iunlockput(ip);
```

Figure 2.4 – sys_unlink in kernel/sysfile.c

```
357   int last = (ip->ref == 1 && ip->valid && ip->nlink == 0);
358   uint dev = ip->dev, inum = ip->inum;
359
360   if (last) {
361     // ip->ref == 1 means no other process can have ip locked.
362     acquiresleep(&ip->lock);
363     release(&itable.lock);
364
365     itrunc(ip); // free the data blocks (type stays nonzero on disk)
366     ip->valid = 0;
367
368     releasesleep(&ip->lock);
369
370     acquire(&itable.lock);
371   }
372
373   ip->ref--;
374   release(&itable.lock);
375
376   if (last)
377     ifree(dev, inum); // now clear type on disk: inum becomes allocatable
```

Figure 2.5 – iput in kernel/fs.c, where the blocks are freed

```
32 struct dinode {
33   short type;               // File type
34   short major;              // Major device number (T_DEVICE only)
35   short minor;              // Minor device number (T_DEVICE only)
36   short nlink;              // Number of links to inode in file system
37   uint size;                // Size of file (bytes)
38   uint addrs[NDIRECT + 1]; // Data block addresses
39 };
```

Figure 2.6 – The on-disk inode, struct dinode in kernel/fs.h

Observations:

1. **After `ln` and after `rm`.** `ln README r2` did not copy anything: both names show the same inode 2, the same size, and the link count went from 1 to 2. After `rm README`, the name `README` is gone (`mystat` can no longer find it), but `r2` still shows inode 2 with the full 2441 bytes, and `wc` confirms the content is intact; only the link count dropped back to 1. What changed is the set of names and the link count; what stayed the same is the inode number, the size and the

data.

2. **Where the link count is decremented, and when the blocks are freed.** In `sys_unlink` (`kernel/sysfile.c`), line 236 overwrites the directory entry with zeros, which removes the name, and line 244, `ip->nlink--`, decrements the inode's link count. The data blocks are not freed there. They are freed in `iput()` (`kernel/fs.c`): only when the link count is 0 *and* this is the last in-memory reference to the inode (`ref == 1`, so no process still has the file open) does `itrunc()` on line 365 free the data blocks, after which `ifree()` marks the inode itself as free. So deleting a file that some program still has open keeps its data until the file is closed.
3. **Fields of an xv6 inode.** `struct dinode` has only `type`, `major` and `minor` (device numbers, used only for devices), `nlink`, `size` and `addrs[NDIRECT + 1]`, the addresses of the data blocks. There is no owner, no group, no permission bits and no timestamps, and no inode type for symbolic links, which is why xv6 has neither permissions nor `ln -s`.

3 Part 3. Measure the limits

```
$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ" kernel/fs.h
5:#define BSIZE   1024 // block size
26:#define NDIRECT   12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
```

Figure 3.1 – The limits defined in kernel/fs.h

```
$ make clean ; make fs.img 2>&1 | grep nmeta
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
```

Figure 3.2 – The disk layout printed by mkfs

```
$ mkdir averyveryverylongname
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
[... 25 entries omitted ...]
a              1 28 32
averyveryveryl 1 29 32
```

Figure 3.3 – A name longer than DIRSIZ

Observations:

1. **The long name.** `ls` shows `averyveryveryl`: only the first 14 characters were kept. The constant is `DIRSIZ = 14`, the size of the name field in `struct dirent`; `dirlink()` copies the name with `strncpy(de.name, name, DIRSIZ)` (`kernel/fs.c`, line 640), which silently cuts everything after 14 characters.
2. **The largest file.** An inode has `NDIRECT = 12` direct block addresses plus one indirect block. The indirect block is a block of `BSIZE = 1024` bytes full of 4-byte block numbers, so it holds `NINDIRECT` $= 1024/4 = 256$ more addresses. A file can therefore use at most $12 + 256 = 268$

blocks (MAXFILE), and the largest file is

$$268 \times 1024 = 274\,432 bytes \approx 268 KB.$$

3. **The disk layout.** The whole disk has 2000 blocks of 1 KB (FSSIZE = 2000). 47 of them are metadata: 1 boot block, 1 superblock, 31 log blocks (a header plus LOGBLOCKS = 30 for the journal), 13 inode blocks and 1 bitmap block. The remaining 1953 blocks hold data. The 13 inode blocks follow from the sizes: a struct dinode is 64 bytes, so 16 fit in a 1024-byte block, and $200/16 + 1 = 13$ blocks hold the 200 inodes that mkfs creates.

4 Part 4. Write find

find starts from user/ls.c: it opens a directory, checks with fstat that it really is one, and reads it as a sequence of 16-byte struct dirent records. For every used entry it builds the full path in a buffer (path/name), calls stat on it, prints the path if the last component equals the name being searched, and calls itself recursively if the entry is a directory. Entries with inum == 0 are free slots (for example left by rm) and are skipped, and so are . and .., otherwise the walk would loop forever. Every directory is closed before find returns, because a process may have only 16 open files (NOFILE) and the recursion keeps one directory open per level. The optional -n argument adds the inode number to each line.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

int shownum;

void
find(char *path, char *name)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(path, O_RDONLY)) < 0){
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }
  if(fstat(fd, &st) < 0 || st.type != T_DIR){
    fprintf(2, "find: %s is not a directory\n", path);
    close(fd);
    return;
  }
  if(strlen(path) + 1 + DIRSIZ + 1 > sizeof buf){
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }

  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';
```

```
  while(read(fd, &de, sizeof(de)) == sizeof(de)){
    if(de.inum == 0)
      continue;
    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = 0;
    if(strcmp(p, ".") == 0 || strcmp(p, "..") == 0)
      continue;
    if(stat(buf, &st) < 0){
      fprintf(2, "find: cannot stat %s\n", buf);
      continue;
    }
    if(strcmp(p, name) == 0){
      if(shownum)
        printf("%s %d\n", buf, st.ino);
      else
        printf("%s\n", buf);
    }
    if(st.type == T_DIR)
      find(buf, name);
  }
  close(fd);
}

int
main(int argc, char *argv[])
{
  if(argc == 4 && strcmp(argv[3], "-n") == 0)
    shownum = 1;
  else if(argc != 3){
    fprintf(2, "usage: find <dir> <name> [-n]\n");
    exit(1);
  }
  find(argv[1], argv[2]);
  exit(0);
}
```

Figure 4.1 – user/find.c

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
```

Figure 4.2 – Testing find on a fresh disk

```
$ find . r2 -n
./a/b/r2 30
./a/r2 31
./r2 3
```

```
$ mystat cat r2 a/r2 a/b/r2
cat: file, inode 3, links 2, size 35904 bytes
r2: file, inode 3, links 2, size 35904 bytes
a/r2: file, inode 31, links 1, size 2 bytes
a/b/r2: file, inode 30, links 1, size 2 bytes
$ find
usage: find <dir> <name> [-n]
$ find README r2
find: README is not a directory
```

Figure 4.3 – Stretch: inode numbers with -n, and the error cases

Results:

1. **The test.** `find . r2` printed the three expected paths and `find a b` printed `a/b`, exactly as in the assignment.
2. **Stretch.** With `-n`, `./r2` has inode 3, the same inode as `cat`, and `mystat` shows both names with 2 links and the size of the `cat` program: `ln cat r2` made `./r2` a second name for the `cat` file. The other two `r2` files (inodes 30 and 31) are separate 2-byte files created with `echo`.

Conclusions

This lab showed the xv6 file system from the inside. A file is an inode, a fixed-size record that holds its type, link count, size and the addresses of its data blocks, while a name lives only in a directory, which is itself an ordinary file filled with 16-byte entries that pair a name with an inode number. Because of this split, a hard link is just one more entry pointing to the same inode, and removing a name only decrements the link count. The `mystat` experiment made this visible, and the kernel code confirmed that the data blocks are freed only when no name and no open file refers to the inode any more.

Reading `kernel/fs.h` showed how small and fixed the design is: 1 KB blocks, 14-character names, 12 direct and 256 indirect block addresses, which limit a file to 274 432 bytes, and a 2000-block disk of which 47 blocks are metadata. Writing `find` combined all of this: reading a directory is reading its entries one by one, every entry is turned into a path and inspected with `stat`, and the tree is walked by recursion, while taking care to skip free slots, `.` and `..`, and to close every directory because a process can hold only 16 open files. The same ideas, with more fields and bigger limits, are what ext4 does on Linux.